

AI Engineering — Final Project

AI-Powered Real Estate Property Triage System

Student Project Guideline

Technologies covered in this project:

n8n · LangChain · LangGraph · Llama.cpp · PyTorch
NeMo Guardrails · Gemini / GPT-5 · Ollama · AWS EC2

Submission deadline: [set by instructor]

Team size: 2–3 students

Table of Contents

1. Project Overview

This final project asks you to design and build a complete, production-style AI system from end to end. You will work as a team to construct a multi-component pipeline that automates the intake and initial evaluation of real estate property listings for a fictional agency.

1.1 The Real-World Scenario

A real estate agency receives dozens of new property submissions every day. Each submission contains a written description and a set of photographs. The agency needs to:

- Validate that the submission is a genuine property listing (not spam or off-topic).
- Identify the property type, condition, and key features from the text.
- Classify each uploaded image by room type and assign a condition score.
- Retrieve similar past listings from an internal knowledge base to aid the agent.
- Route the processed listing to the correct team (residential vs. commercial).
- Produce a structured, ready-to-publish listing brief.

Your system automates this entire pipeline. Every component must work together in an integrated, testable flow.

Why this scenario?

Real estate listing intake is a realistic, multi-modal workflow. It combines text understanding, image analysis, retrieval-augmented generation, safety filtering, and agent-based reasoning — all in a single coherent product. Unlike toy demos, every design decision you make has a visible effect on the output quality.

1.2 Learning Objectives

By completing this project you will demonstrate the ability to:

1. Design and deploy a multi-service AI system on cloud infrastructure (AWS EC2).
2. Build and configure n8n automation flows with AI agent nodes and external HTTP calls.
3. Implement a RAG pipeline using LangChain, Llama.cpp, and a vector database.
4. Write and fine-tune a PyTorch image classification service.
5. Design and iterate on prompt engineering across at least five distinct system components.
6. Implement input and output safety guardrails using NeMo Guardrails or a custom rule engine.
7. Build a multi-step reasoning agent with LangGraph.
8. Integrate a local Ollama server with a web-based conversational UI.

2. System Architecture

The system is composed of four distinct layers. Each layer communicates with the next via HTTP. The diagram below shows the complete architecture.

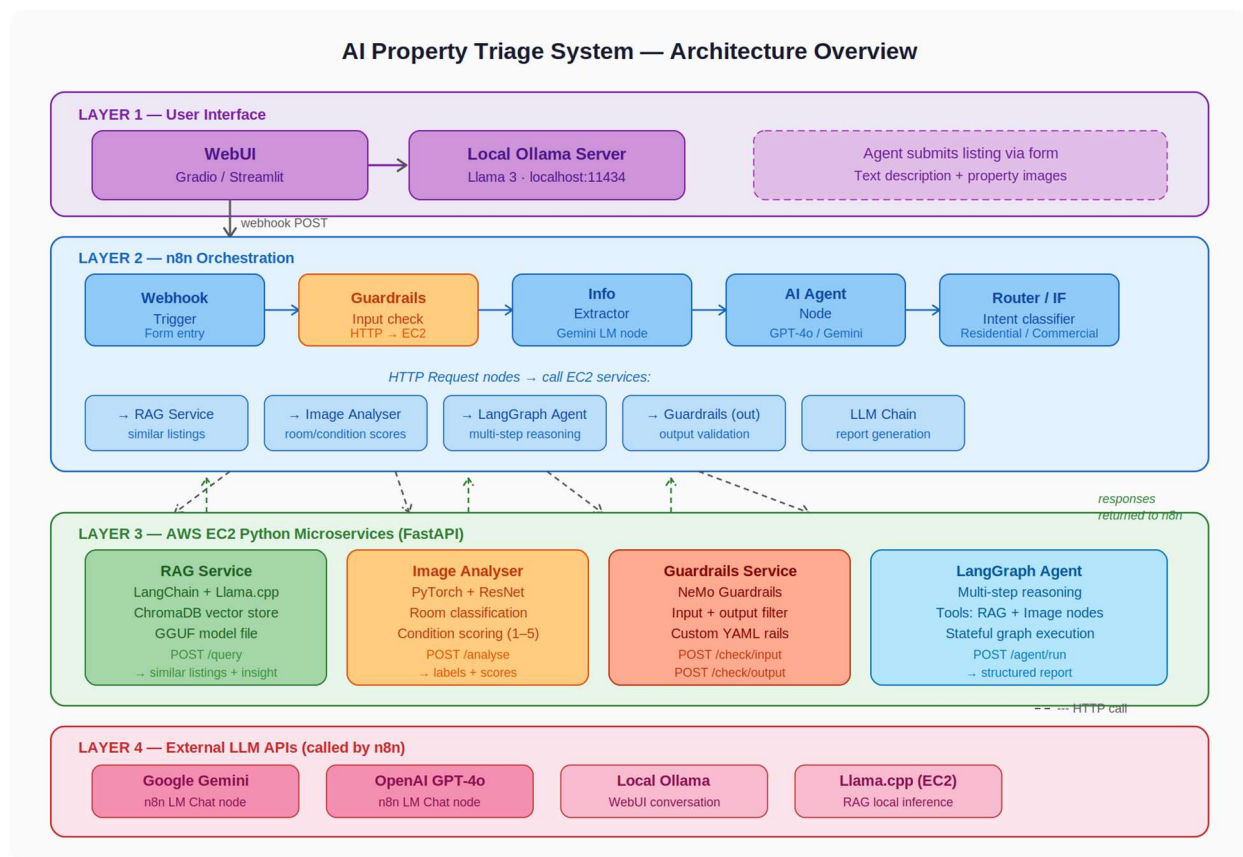


Figure 1 — Full system architecture overview

2.1 Layer Descriptions

Layer 1 — User Interface

A Gradio or Streamlit web application that serves two purposes. First, it provides a conversational assistant sidebar powered by a local Ollama server running Llama 3. The agent can ask questions about the property market, and the model responds in real time. Second, it contains a submission form where a listing agent uploads a property description and images. On submission, the form sends a POST request to the n8n webhook.

Layer 2 — n8n Orchestration

The n8n flow is the central orchestrator. It receives the webhook, calls the guardrails service to validate the input, extracts structured fields using an n8n Information Extractor node (backed by Gemini), passes the data to an AI Agent node for intent classification and enrichment, dispatches parallel HTTP requests to the EC2 services, collects the results, runs the output through an LLM Chain to produce the final report, and routes the result.

Layer 3 — AWS EC2 Microservices

Four independent Python FastAPI services run on EC2 instances. Each service is containerised with Docker and exposes a single well-defined HTTP endpoint. The services are: the RAG service, the Image Analyser, the Guardrails service, and the LangGraph Agent.

Layer 4 — External LLM APIs

The n8n AI Agent node connects to either Google Gemini or OpenAI GPT-4o via the respective n8n LM Chat nodes. The local Ollama server serves the WebUI. The RAG service uses Llama.cpp for local inference with a GGUF model file stored on the EC2 instance.

3. n8n Flow Specification

The diagram below shows the full n8n flow you must build. Each numbered node is described in detail below.

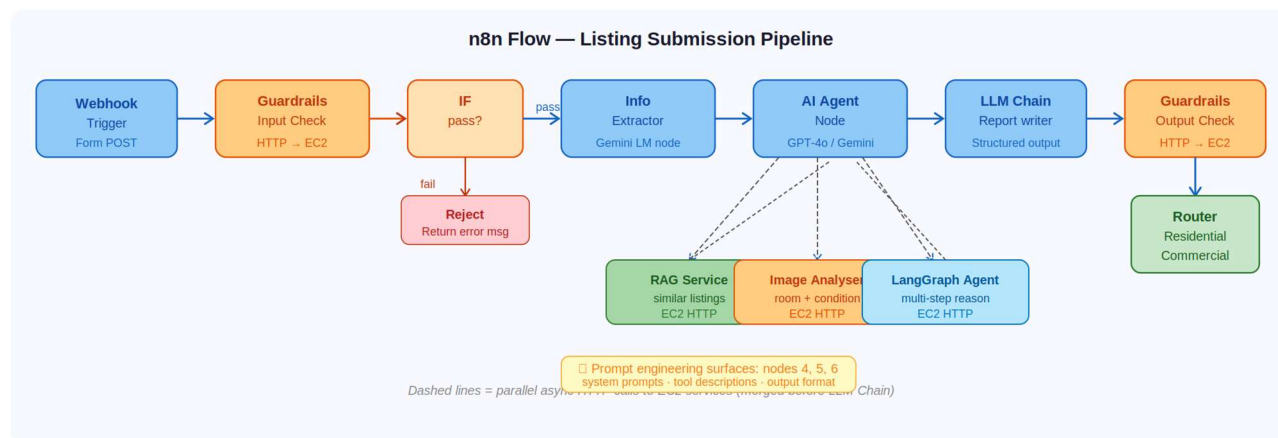


Figure 2 — n8n flow: listing submission pipeline

3.1 Node Descriptions

Node 1 — Webhook Trigger

Receives the form POST from the WebUI. Parses the incoming JSON body containing the property description text and image URLs. Passes the payload to node 2.

Node 2 — Guardrails Input Check

Sends an HTTP POST to the Guardrails service on EC2 with the raw input text. Receives a pass/fail response. If fail, the flow branches to the rejection path. If pass, continues to node 3.

Node 3 — IF (Pass/Fail Branch)

A standard n8n IF node. Routes to the rejection response node on failure, or continues to the Information Extractor on success.

Node 4 — Information Extractor (Prompt Engineering Surface)

An n8n Information Extractor node backed by a Gemini LM node. Extracts structured fields: property type, location, price, number of rooms, key features, and any mentioned certifications. Students must design the attribute definitions and the system prompt template. This is a major prompt engineering task.

Prompt Engineering Task — Node 4

Write the systemPromptTemplate for the Information Extractor. It must instruct the model to extract only facts present in the text, handle missing fields gracefully, and return consistent types. Iterate at least 5 times, documenting failures and improvements each time.

Node 5 — AI Agent Node (Prompt Engineering Surface)

An n8n AI Agent node connected to GPT-4o or Gemini. Receives the structured extracted fields and the image URLs. Dispatches tool calls to the three EC2 services (RAG, Image Analyser,

LangGraph). The agent system prompt defines its role, available tools, and output format. Students write and tune this prompt.

Prompt Engineering Task — Node 5

Write the AI Agent system prompt. Define the agent role (senior property analyst), the tools it can call, and the structured JSON format it must return. The quality of tool descriptions directly affects whether the agent selects the correct tool. Iterate at least 5 times.

Node 6 — LLM Chain (Prompt Engineering Surface)

Receives the merged outputs from the AI Agent and the EC2 service calls. Produces the final listing brief in a specified HTML or Markdown format. The prompt must instruct the model to synthesise retrieved context, image scores, and extracted facts into a coherent, publishable brief without inventing missing information.

Prompt Engineering Task — Node 6

Write the LLM Chain prompt that produces the final listing brief. The model must integrate three sources of information: the extracted fields, the image condition scores, and the similar-listings context from the RAG service. Define the exact output structure with a JSON schema in a Structured Output Parser node.

Node 7 — Guardrails Output Check

Sends the generated report to the Guardrails service output endpoint. Validates that the report contains no false legal claims, price guarantees, or fabricated facts. If the output fails, the flow flags it for human review instead of returning it to the WebUI.

Node 8 — Router

Reads the `property_type` field from the structured output. Routes to the residential team webhook if the type is apartment, house, or villa; routes to the commercial team webhook for office, retail, or industrial listings.

4. EC2 Microservices

Each service must be implemented as a Python FastAPI application, containerised with Docker, and deployed on an AWS EC2 instance. The diagram below shows the internal architecture of each service.

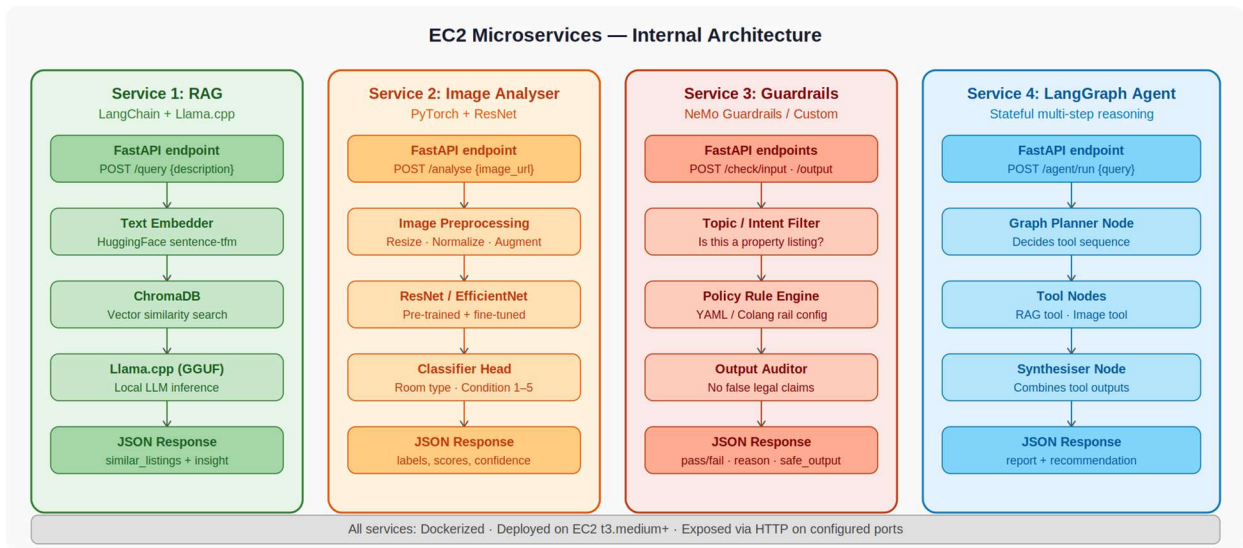


Figure 3 — EC2 services: internal architecture

4.1 Service 1 — RAG Service

Endpoint	POST /query
Input	{ "description": "<listing text>" }
Output	{ "similar_listings": [...], "insight": "<generated text>" }
Stack	LangChain · Llama.cpp · ChromaDB · HuggingFace embeddings

Build a FastAPI server that receives a property description, embeds it using a sentence-transformer model, queries a ChromaDB vector store for the top-3 most similar past listings, injects the retrieved context into a LangChain prompt, and generates a short insight using Llama.cpp with a GGUF model file. The vector store must be pre-populated with at least 20 synthetic property listings before deployment.

Prompt Engineering Task — RAG Retrieval Prompt

Write the LangChain prompt that instructs Llama.cpp to use the retrieved context. The model must cite which similar listing it is drawing from, and must not fabricate details not present in the retrieved documents. Iterate at least 5 times on prompt structure, context injection format, and instruction phrasing.

4.2 Service 2 — Image Analyser

Endpoint	POST /analyse
Input	{ "image_url": "<url>" }
Output	{ "room_type": "kitchen", "condition_score": 4, "confidence": 0.91 }
Stack	PyTorch · torchvision · ResNet-50 or EfficientNet-B0

Train or fine-tune a CNN to classify property images into the following room types: kitchen, bathroom, living room, bedroom, exterior, and other. Add a second output head for a condition score from 1 (poor) to 5 (excellent). The base model should be a pre-trained ResNet-50 or EfficientNet-B0. Students are responsible for sourcing or synthesising a training dataset of at least 200 labelled images, writing the training loop, and implementing inference with a confidence threshold below which the service returns an uncertain label.

Implementation Note

You do not need to train from scratch. Use transfer learning: freeze the convolutional backbone, replace the classifier head, and fine-tune on your dataset. Document your data augmentation strategy and report final test set accuracy in your submission.

4.3 Service 3 — Guardrails Service

Endpoint (input)	POST /check/input
Endpoint (output)	POST /check/output
Input	{ "text": "<text to check>" }
Output	{ "pass": true/false, "reason": "<if fail>", "safe_text": "<if output>" }
Stack	NeMo Guardrails · FastAPI · YAML/Colang rail configs

Build a dual-endpoint guardrails service. The input endpoint checks whether the submitted text is a genuine property listing in the expected language, rejecting spam, offensive content, and off-topic submissions. The output endpoint checks the AI-generated report for false legal claims, invented prices, and fabricated certifications.

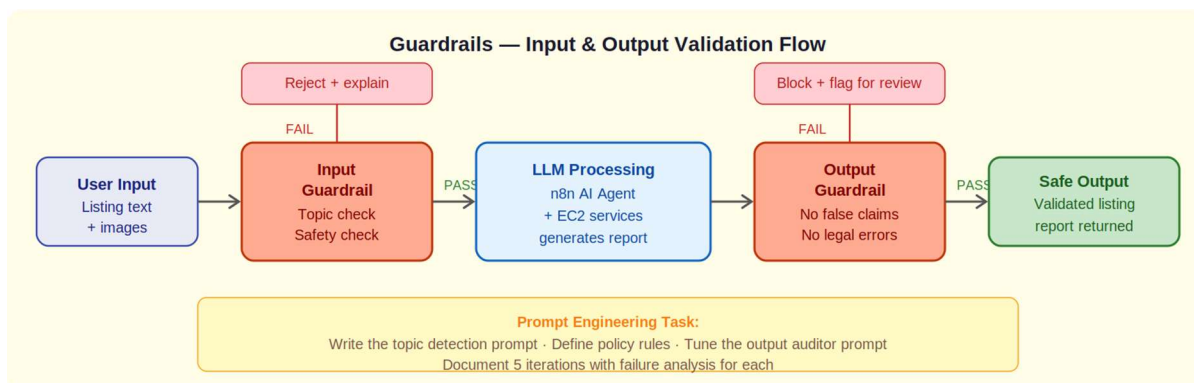


Figure 4 — Guardrails input and output validation flow

Prompt Engineering Task — Guardrails Rail Prompts

The NeMo Guardrails rail configuration includes a topic detection prompt and a policy enforcement prompt. Both are prompts, not code. Write and iterate on these prompts to minimise false positives (rejecting valid listings) while maintaining strong protection against invalid inputs and unsafe outputs. Iterate at least 5 times on each.

4.4 Service 4 — LangGraph Agent

Endpoint	POST /agent/run
Input	{ "query": "<complex question about the listing>" }
Output	{ "answer": "...", "tools_used": [...], "reasoning_steps": [...] }
Stack	LangGraph · FastAPI · Llama.cpp or external LLM API

Build a stateful LangGraph agent with three nodes: a planner node that determines which tools to call, a tool execution node that can invoke both the RAG service and the Image Analyser, and a synthesiser node that combines tool outputs into a final answer. The agent is designed to answer complex multi-step questions such as: What renovation work would be needed to bring this property to a condition score of 5? Which rooms in the uploaded images need attention?

Prompt Engineering Task — LangGraph Tool Descriptions

The LangGraph agent selects tools based on their text descriptions. Write clear, precise tool descriptions for the RAG tool and the Image Analyser tool. Vague descriptions will cause the agent to call the wrong tool or skip useful ones. Iterate at least 5 times, testing each version with the same set of 10 benchmark queries.

5. WebUI and Local Ollama Integration

Build a Gradio or Streamlit application with two tabs or sections:

5.1 Conversational Assistant Tab

A chat interface that sends user messages to a locally running Ollama server (Llama 3 or Mistral). The system prompt grounds the model as a real estate assistant, preventing it from answering off-topic questions. Students tune this system prompt as part of the prompt engineering deliverable.

Prompt Engineering Task — Ollama System Prompt

Write the system prompt that instructs the local Ollama model to act as a knowledgeable real estate assistant. It must refuse off-topic queries politely, stay factual, and not invent prices or legal advice. Iterate at least 5 times, testing edge cases such as off-topic questions, requests for legal advice, and attempts to override the system prompt.

5.2 Listing Submission Tab

A form with fields for: listing description text, image URLs (comma-separated or file upload), and listing agent name. On submission, the form POST is sent to the n8n webhook. The UI should display the structured report returned by n8n when the flow completes, including the image condition scores and similar-listing recommendations.

6. Prompt Engineering Log

The Prompt Engineering Log is a graded deliverable worth 25% of the total project score. For each of the five prompt engineering surfaces listed below, you must submit a structured log documenting your iteration process.

#	Surface	Component
1	n8n Information Extractor	Node 4 — systemPromptTemplate for structured field extraction
2	n8n AI Agent	Node 5 — agent system prompt and tool descriptions
3	LangChain RAG Retrieval	EC2 Service 1 — context injection and citation instructions
4	Guardrails Rail Prompts	EC2 Service 3 — topic detection and output auditor prompts
5	Local Ollama System Prompt	WebUI — real estate assistant grounding and refusal behaviour

6.1 Log Format (per surface)

Each prompt engineering log entry must follow this structure:

Version 1 — Baseline

Write your first attempt at the prompt without overthinking it. Run it on your test cases and record the outputs.

Versions 2 and 3 — Targeted Iterations

Identify a specific failure mode from the previous version. Describe the failure in one sentence. Modify the prompt to address it. Record the new outputs and assess whether the failure was resolved and whether any regressions appeared.

Versions 4 and 5 — Refinement

Continue iterating. By version 5 you should be able to articulate what you learned about how this model responds to instructions, what phrasing patterns work, and what consistently fails.

Final Entry

State the final prompt, justify each design decision in the prompt, and report the pass rate on your test suite (minimum 10 test cases per surface).

7. Deliverables

7.1 Code Submission

- n8n flow exported as a JSON file (importable into n8n).
- Source code for all four EC2 services, each in its own directory with a Dockerfile and requirements.txt.
- WebUI source code (app.py or equivalent) with a README for running locally.
- Pre-population script for the ChromaDB vector store (minimum 20 listings).
- PyTorch model checkpoint (.pth) for the Image Analyser, or a training script and instructions to reproduce the training.

7.2 Documentation

- Prompt Engineering Log (PDF or DOCX) — see Section 6.
- Architecture diagram (your own version, not a copy of Figure 1) annotating your specific design decisions.
- Deployment notes: EC2 instance types used, ports configured, and any deviations from the specification.

7.3 Demo

- A 5–8 minute video recording demonstrating: a successful end-to-end listing submission, a listing that fails the input guardrail, a listing that fails the output guardrail, and a conversational exchange with the Ollama assistant.

Submission Format

Submit all files as a single ZIP archive named TEAMNAME_FinalProject.zip. The archive must contain: /code (all source), /docs (all documentation), /demo (video file or link), and a top-level README.md with setup and run instructions.

8. Technology Summary

Component	Technology	Deployment
WebUI	Gradio or Streamlit	Localhost
Conversational LLM	Ollama + Llama 3 / Mistral	Localhost
n8n Orchestration	n8n (self-hosted or cloud)	n8n
LM nodes (n8n)	Google Gemini / GPT-4o	External API
RAG Service	LangChain + Llama.cpp + ChromaDB	AWS EC2
Image Analyser	PyTorch + ResNet / EfficientNet	AWS EC2
Guardrails Service	NeMo Guardrails + FastAPI	AWS EC2
LangGraph Agent	LangGraph + FastAPI	AWS EC2
Vector Store	ChromaDB	AWS EC2 (local)
Model Files	GGUF (Llama 3 / Mistral 7B)	AWS EC2 (EBS)

9. Grading Rubric

Criterion	Weight	Excellent (90-100)	Satisfactory (70-89)	Insufficient (<70)
n8n Flow (20%)	20%	All nodes implemented, guardrails integrated, router working, parallel EC2 calls wired correctly.	Main flow works end-to-end with minor issues in routing or parallel calls.	Flow incomplete or cannot process a submission end-to-end.
EC2 Services (25%)	25%	All four services deployed, Dockerized, endpoints return correct schema, services handle error cases.	At least three services deployed and functional. Minor schema issues.	Fewer than three services functional or not deployed on EC2.
Image Analyser (10%)	10%	Model trained, achieves >75% accuracy on test set, handles uncertain images gracefully.	Model functional, accuracy 55-75%, basic inference works.	Model not trained or not integrated into the FastAPI service.
Guardrails (10%)	10%	Both input and output rails working, YAML configs well-designed, less than 5% false positive rate on valid listings.	Rails functional, some false positives, output rail may miss edge cases.	Guardrails not implemented or always pass/always fail.
Prompt Engineering Log (25%)	25%	5 surfaces covered, 5 iterations each, clear failure analysis, measurable improvement demonstrated.	4 surfaces, 3-4 iterations each, some failure analysis present.	Fewer than 3 surfaces, fewer than 3 iterations, no failure analysis.
WebUI + Ollama (10%)	10%	Both tabs functional, Ollama system prompt well-tuned, form submits to n8n, report displayed correctly.	WebUI functional, Ollama responds, minor display issues with report.	WebUI missing or Ollama not integrated.

10. Optional Extensions

Teams that complete all required components and are seeking additional challenge may implement one or more of the following extensions. Extensions are evaluated for extra credit.

Feedback Loop and Active Learning

Add a review queue to the WebUI. When the output guardrail flags a report as uncertain, store the listing and the generated report in a queue visible to a human reviewer. Add a simple annotation interface where the reviewer can mark the report as correct or incorrect. Use the annotated data to periodically retrain the Image Analyser classifier.

Multilingual Guardrail

Extend the input guardrail to detect submissions in languages other than the expected language (Hebrew or English, depending on your deployment). Reject submissions in unexpected languages with a polite, localised error message.

Managed Vector Store

Replace the local ChromaDB instance with a managed vector store such as Pinecone or Weaviate. Benchmark retrieval quality (precision@3) before and after the migration and include the comparison in your documentation.

Monitoring Dashboard

Add a third tab to the WebUI showing real-time statistics: number of listings processed, guardrail hit rate (inputs rejected), average image condition score, average n8n execution time, and a log of the last 10 processing events.

11. Setup and Deployment Guide

11.1 Prerequisites

- AWS account with permission to launch EC2 instances (t3.medium or larger recommended).
- Python 3.10+ installed locally for development.
- Docker Desktop installed locally for testing containers before deployment.
- n8n account (n8n.cloud free tier is sufficient) or self-hosted n8n.
- Google AI Studio API key (for Gemini) or OpenAI API key (for GPT-4o).
- Ollama installed locally: <https://ollama.ai>

11.2 Local Development Order

9. Install Ollama and pull the Llama 3 model: `ollama pull llama3`
10. Build and test each EC2 service locally with Docker before deploying.
11. Create the ChromaDB vector store and populate it with synthetic listings.
12. Test each FastAPI service endpoint with curl or Postman.
13. Build the n8n flow and point the HTTP nodes at localhost during testing.
14. Build the WebUI and confirm it connects to Ollama and submits to n8n.

11.3 EC2 Deployment Checklist

- Launch an EC2 instance (t3.medium minimum, t3.large recommended for GPU-less inference).
- Install Docker on the instance: `sudo apt-get install docker.io`
- Copy the service directory to the instance with scp or git clone.
- Build the Docker image: `docker build -t service-name .`
- Run the container with the correct port mapping: `docker run -p 8000:8000 service-name`
- Configure the EC2 security group to allow inbound TCP on the service port from the n8n IP.
- Update the n8n HTTP node URLs from localhost to the EC2 public IP.

Security Note

Do not expose EC2 service ports to 0.0.0.0/0. Restrict inbound rules to the IP addresses of your n8n instance and your development machine only. Never commit API keys or AWS credentials to your repository.

Appendix — Recommended Resources

LangChain and LangGraph

- LangChain documentation: <https://python.langchain.com>
- LangGraph documentation: <https://langchain-ai.github.io/langgraph>
- LangChain RAG tutorial: <https://python.langchain.com/docs/tutorials/rag>

Llama.cpp

- llama-cpp-python (Python bindings): <https://github.com/abetlen/llama-cpp-python>
- GGUF model files: <https://huggingface.co/models?search=gguf>

PyTorch

- Transfer learning tutorial:
https://pytorch.org/tutorials/beginner/transfer_learning_tutorial.html
- torchvision model zoo: <https://pytorch.org/vision/stable/models.html>

NeMo Guardrails

- NeMo Guardrails documentation: <https://docs.nvidia.com/nemo/guardrails>
- Colang language reference: https://docs.nvidia.com/nemo/guardrails/colang_2

n8n

- n8n documentation: <https://docs.n8n.io>
- n8n AI nodes overview: <https://docs.n8n.io/integrations/builtin/cluster-nodes/root-nodes/>

Ollama

- Ollama documentation: <https://ollama.ai/docs>
- Gradio documentation: <https://www.gradio.app/docs>

Good luck. Build something you would be proud to show in an interview.